Light

Programming — Testing & Robustness

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

Assessment Hub | Notes / Textbook | Long-Answer Practice

Assessment Hub
Long-Answer
Notes Index
What you must know
Toolkit
1. Testing and test data
2. Errors in programs
3. Robust programming
Worked examples
Quick reference
Typical pitfalls

AQA GCSE Computer Science — **Programming Testing and Robustness (8525 Paper 1)**. This topic covers testing programs with different types of test data, identifying and correcting syntax and logic errors, and writing robust programs using data validation and authentication routines.

Exam questions typically ask you to **identify** types of test data, **select and justify** suitable test data, **identify and categorise** errors as syntax or logic errors, **correct** errors in code, and **write** validation and authentication routines. Questions range from 1-mark identification to multi-mark code writing tasks.

Why testing and robustness matter

Programs rarely work perfectly first time. **Testing** is the process of checking that a program works correctly by running it with different inputs and checking the outputs are as expected. **Robust** programs are designed to handle unexpected inputs gracefully without crashing or producing incorrect results.

Professional programmers spend significant time testing their code and building in validation checks. In the exam, you need to understand how to choose appropriate test data, identify different types of errors, and write code that validates user input and authenticates users.

- **Normal data:** Typical, valid inputs the program should accept
- **Boundary data:** Values at the edges of valid ranges
- **Erroneous data:** Invalid inputs the program should reject
- **Syntax error:** Code that breaks the rules of the programming language
- **Logic error:** Code that runs but produces incorrect results

What you must know

This checklist covers the key exam-visible concepts from the AQA specification. You should be able to explain each point and demonstrate each skill.

- **Understand** what is meant by testing in the context of algorithms and programs.
- **Understand** what test data is and **describe** the following types of test data: normal (typical), boundary (extreme), and erroneous data.
- **Be able to select and justify** the choice of suitable test data for a given problem.
- **Understand** that there are different types of error: syntax error and logic error.
- **Be able to identify and categorise** errors within algorithms and programs.
- **Be able to correct** errors within algorithms and programs.
- **Be able to write** simple data validation routines.
- **Be able to write** simple authentication routines.

> **Specification note.** AQA 8525 assesses testing and robustness on Paper 1 (Computational thinking and programming skills). Validation routines include checking minimum string length, checking if a string is empty, and checking data lies within a given range. Authentication routines use plain text usernames and passwords only — encryption is not required at GCSE.

Toolkit

Test data types at a glance

Type	Description	Example (range 1—10)	Purpose
Normal	Typical, valid data within the expected range	5, 7, 3	Check program works for typical inputs
Boundary	Values at the edges of the valid range	0, 1, 10, 11	Check boundaries are handled correctly
Erroneous	Invalid data that should be rejected	-5, 100, "abc"	Check program handles invalid input

Error types at a glance

Error type	Description	When detected	Example
Syntax error	Code breaks the rules of the programming language	When the program is translated (compiled/interpreted)	Missing colon, misspelled keyword, unclosed bracket
Logic error	Code runs but produces incorrect results	When the program runs (runtime)	Using + instead of -, wrong comparison operator

Boundary data rule

For a valid range of **1 to 10** (inclusive), boundary test data should include:

- **0** — just below the lower boundary (should be rejected)
- **1** — at the lower boundary (should be accepted)
- **10** — at the upper boundary (should be accepted)
- **11** — just above the upper boundary (should be rejected)

This tests both sides of each boundary to ensure the conditions are correct.

Validation vs Authentication

Concept	Purpose	Examples
Validation	Checking that data entered is reasonable/sensible/in the correct format	Range check, length check, presence check, type check
Authentication	Verifying that a user is who they claim to be	Username and password check

Common validation checks

Check type	What it checks	Python example
Range check	Value is within allowed minimum and maximum	`if num >= 1 and num <= 10:`
Length check	String has required minimum/maximum length	`if len(password) >= 8:`

Check type	What it checks	Python example
Presence check	Data has been entered (not empty)	`if name != "":`
Type check	Data is of the correct type	`if age.isdigit():`

1. Testing and test data

Testing is the process of running a program with known inputs and checking that the outputs are correct. Testing helps find errors (bugs) in programs before they are released to users.

1.1 Why we test programs

Programs are tested to:

- Check that the program produces correct outputs for valid inputs
- Check that the program handles invalid inputs appropriately
- Find and fix errors before the program is used by real users
- Ensure the program meets its requirements

Testing cannot prove a program is completely correct — it can only show that no errors were found with the test data used. This is why choosing good test data is essential.

1.2 Test data

Test data is the input values used to test a program. Good test data should include a variety of inputs that test different aspects of the program.

Normal (typical) data

Normal data is typical, valid data that the program should accept and process correctly. It represents the kind of input the program will receive during normal use.

> **Example:** If a program accepts ages between 1 and 120, normal test data might be: 25, 45, 67
> These are typical ages that should be processed correctly.

Boundary (extreme) data

Boundary data tests the edges of valid ranges. Boundary errors are common because programmers often make mistakes with comparison operators (using < instead of <= for example).

> **Example:** If the valid range is 1 to 10:
> - **0** — just below lower boundary (should be rejected)
> - **1** — at lower boundary (should be accepted)
> - **10** — at upper boundary (should be accepted)
> - **11** — just above upper boundary (should be rejected)

Testing both sides of each boundary helps ensure the comparison operators are correct. If testing only 1 and 10, you might miss an error where the condition is `num > 1` instead of `num >= 1`.

Erroneous data

Erroneous data is invalid data that the program should reject. This tests that the program handles bad input gracefully without crashing or producing incorrect results.

Example: If a program expects a positive integer:

- **-5** — negative number
- **"abc"** — text instead of a number
- **3.5** — decimal instead of integer
- **""** — empty input

1.3 Creating a test plan

A **test plan** (or test table) documents the test data to be used, the expected results, and the actual results when the tests are run.

Test	Test data	Type	Expected result	Actual result	Pass/Fail
1	5	Normal	Accepted	Accepted	Pass
2	1	Boundary	Accepted	Accepted	Pass
3	0	Boundary	Rejected	Rejected	Pass
4	"abc"	Erroneous	Rejected	Crash	Fail

When a test fails, the programmer must find and fix the error, then re-run all tests to ensure the fix hasn't broken anything else.

1.4 Selecting and justifying test data

When selecting test data, you should:

- Include at least one example of each type (normal, boundary, erroneous)
- Test all boundaries (both edges of each valid range)
- Consider different types of invalid input
- Be able to justify why each piece of test data was chosen

Justification patterns for exam answers

When asked to justify your choice of test data, use these patterns:

- **Normal data justification:** "This is a typical valid input that tests the program works correctly under normal conditions."
- **Boundary data justification:** "This tests the edge of the valid range to ensure the comparison operators are correct."
- **Erroneous data justification:** "This invalid input tests that the program handles bad data without crashing."

1.5 Iterative testing

Testing is not a one-time activity. **Iterative testing** means testing repeatedly throughout development:

- Test individual parts of the program as they are written
- Test after making any changes or fixes

- Re-run all tests after fixing a bug (to check the fix hasn't broken something else)
- Test with new test data if new requirements are added

A good test plan is a living document that grows as the program develops. When a test reveals a bug, the programmer fixes it and then re-runs all tests to ensure the program still works correctly.

> **Testing tip.** If a program crashes or gives wrong output for erroneous data, this reveals a bug. The program should be modified to validate input and reject invalid data gracefully, then re-tested.

Worked Example 1 — Identifying test data types

Question: A program accepts test scores between 0 and 100. State what type of test data each of the following represents: (a) 50 (b) 0 (c) -5 (d) 100 [4 marks]

Model answer:
(a) 50 — **Normal** data (typical valid score) **(1)**
(b) 0 — **Boundary** data (at the lower boundary) **(1)**
(c) -5 — **Erroneous** data (invalid, below allowed range) **(1)**
(d) 100 — **Boundary** data (at the upper boundary) **(1)**

> **Exam tip:** Boundary data is at the exact edge of the valid range (0 and 100 here). Erroneous data is clearly outside the valid range or wrong type entirely.

Worked Example 2 — Selecting boundary data

Question: A program validates that a number entered is between 5 and 20 inclusive. State four items of boundary test data and for each state whether it should be accepted or rejected. [4 marks]

Model answer:
4 — should be rejected (just below lower boundary) **(1)**
5 — should be accepted (at lower boundary) **(1)**
20 — should be accepted (at upper boundary) **(1)**
21 — should be rejected (just above upper boundary) **(1)**

> **Exam tip:** Always test both sides of each boundary. For range 5—20, test 4, 5, 20, and 21.

Worked Example 3 — Completing a test plan

Question: A program asks the user to enter a month number between 1 and 12. Complete the test plan below by filling in the missing test data and expected results.

[4 marks]

Test data	Type	Expected result
6	Normal	?
?	Boundary (lower)	Accepted
13	?	Rejected
?	Erroneous	Rejected

Model answer:

Test data	Type	Expected result
6	Normal	**Accepted (1)**
1 (1)	Boundary (lower)	Accepted
13	**Boundary (upper) (1)**	Rejected
"January" or -3 (1)	Erroneous	Rejected

Exam tip: 13 is boundary data (just above upper boundary 12), not erroneous. Erroneous would be something completely wrong like text or a very large negative number.

Worked Example 4 — Justifying test data choices

Question: A password must be at least 8 characters long. Give one example of normal, boundary, and erroneous test data for this validation. Justify each choice. [6 marks]

Model answer:
Normal: "MyPassword123" **(1)**
Justification: This is a typical password that clearly meets the minimum length requirement (13 characters), representing normal use. **(1)**
Boundary: "Abcdefgh" (exactly 8 characters) **(1)**
Justification: This tests the exact boundary condition to ensure passwords of exactly 8 characters are accepted. **(1)**
Erroneous: "Short" (5 characters) **(1)**
Justification: This is too short and should be rejected, testing that the validation correctly rejects passwords below the minimum length. **(1)**

> **Exam tip:** When asked to justify, explain WHY this data tests what it's supposed to test. Don't just describe the data.

2. Errors in programs

Errors in programs are called **bugs**. Finding and fixing bugs is called **debugging**. AQA requires you to understand two types of error: syntax errors and logic errors.

2.1 Syntax errors

A **syntax error** occurs when code breaks the rules of the programming language. The code cannot be translated (compiled or interpreted) because the translator does not understand it.

> **Definition:** A syntax error is when the code breaks the grammatical rules of the programming language, preventing the program from being translated.

Syntax errors are detected **before** the program runs, when the code is being translated. The translator will display an error message indicating where the problem is.

Common syntax errors in Python

Error	Incorrect code	Correct code
Missing colon after if/while/for	`if x > 5`	`if x > 5:`
Misspelled keyword	`pritn("Hello")`	`print("Hello")`
Unclosed bracket	`print("Hello"`	`print("Hello")`
Unclosed string	`name = "John`	`name = "John"`
Wrong indentation	`if x > 5:` `print("Big")`	`if x > 5:` `  print("Big")`
Using = instead of ==	`if x = 5:`	`if x == 5:`

2.2 Logic errors

A **logic error** occurs when the code is syntactically correct (it can be translated and run) but produces incorrect results. The program runs, but it doesn't do what was intended.

> **Definition:** A logic error is when the code runs without crashing but produces incorrect or unexpected results because the algorithm is wrong.

Logic errors are detected **during** or **after** the program runs, when you notice the output is wrong. They are harder to find because the program doesn't crash — it just gives wrong answers.

Common logic errors

Error	Incorrect code	Correct code	Problem
Wrong operator	`total = a - b`	`total = a + b`	Subtracts instead of adding
Wrong comparison	`if x > 10:`	`if x >= 10:`	Excludes value 10
Off-by-one error	`for i in range(1, 10):`	`for i in range(1, 11):`	Loops 9 times instead of 10
Condition logic wrong	`if x > 5 and x > 10:`	`if x > 5 and x < 10:`	Wrong comparison direction
Variable not updated	`total = total + num` outside loop	`total = total + num` inside loop	Only adds once, not each iteration

2.3 Comparing syntax and logic errors

Syntax errors

- Break language rules
- Detected by translator
- Program will not run
- Error message shown
- Usually easy to find

Logic errors

- Algorithm is wrong
- Detected by testing
- Program runs but gives wrong results
- No error message
- Can be hard to find

2.4 Finding and correcting errors

To find errors:

- **Syntax errors:** Read the error message carefully — it usually tells you the line number and type of error
- **Logic errors:** Use a trace table to track variable values step by step, or add print statements to see what values variables have during execution

To correct errors:

- Identify exactly what is wrong
- Fix the specific error (don't change things randomly)
- Re-test to confirm the fix works
- Check that fixing one error hasn't introduced another

Using print statements to find logic errors

Adding temporary print statements can help you see what your program is doing:

```
total = 0
for i in range(1, 5):
    print("i =", i)  # Debug: shows loop counter
    total = total + i
    print("total =", total)  # Debug: shows running total
print("Final total:", total)
```

This technique shows you the values of variables at each step, helping you spot where the logic goes wrong. Remove the debug print statements once you've fixed the error.

Using trace tables to find logic errors

For more complex programs, a trace table systematically tracks every variable change:

Line	i	total	Notes
1	—	0	Initialise total
2	1	0	First iteration starts
3	1	1	total = 0 + 1
2	2	1	Second iteration starts
3	2	3	total = 1 + 2

This helps you compare expected values against actual values to pinpoint where the error occurs.

2.5 Common error patterns

Knowing common error patterns helps you find and fix bugs faster:

Off-by-one errors

Loops that run one too many or one too few times.

Example: Using `range(1, 10)` when you want 1 to 10 inclusive (should be `range(1, 11)`).

Operator confusion

Using the wrong operator for the intended operation.

Example: Using = (assignment) instead of == (comparison) in conditions.

Worked Example 5 — Identifying syntax errors

Question: Identify the syntax error in the following Python code and write the corrected line. [2 marks]

```
age = int(input("Enter your age: "))
if age >= 18
    print("You can vote")
```

Model answer:
The syntax error is: missing colon at the end of line 2 **(1)**
Corrected line: `if age >= 18:` **(1)**

Exam tip: In Python, if, elif, else, while, for, and def statements must end with a colon. This is one of the most common syntax errors.

Worked Example 6 — Identifying logic errors

Question: The following program should calculate the average of three numbers, but it contains a logic error. Identify the error and write the corrected line. [2 marks]

```
num1 = int(input("Enter number 1: "))
num2 = int(input("Enter number 2: "))
num3 = int(input("Enter number 3: "))
average = num1 + num2 + num3 / 3
print("The average is:", average)
```

Model answer:
The logic error is: only num3 is divided by 3 (not the total) due to operator precedence **(1)**
Corrected line: `average = (num1 + num2 + num3) / 3` **(1)**

Exam tip: This is a logic error because the code runs without crashing — it just gives the wrong answer. Division happens before addition unless you use brackets.

Worked Example 7 — Categorising errors

Question: For each of the following, state whether it is a syntax error or a logic error. [4 marks]
(a) A program that calculates someone's age but gives an answer 1 year too high.
(b) A program that will not run because of a missing bracket.
(c) A loop that repeats 9 times instead of 10 times.
(d) A program that crashes because `print` is spelled `prnt`.

Model answer:
(a) **Logic error** — the program runs but gives incorrect results **(1)**
(b) **Syntax error** — breaks language rules, program won't run **(1)**
(c) **Logic error** — the loop runs but the wrong number of times **(1)**
(d) **Syntax error** — misspelled keyword, program won't run **(1)**

Exam tip: Ask yourself: "Does the program run?" If no, it's a syntax error. If yes but the output is wrong, it's a logic error.

Worked Example 8 – Finding and fixing multiple errors

Question: The following program should ask for a number and output whether it is positive, negative, or zero. It contains two syntax errors. Identify and correct both errors. [4 marks]

```
num = int(input("Enter a number: "))
if num > 0:
    print("Positive")
elif num = 0:
    print("Zero")
else
    print("Negative")
```

Model answer:
Syntax error 1: Line 4 uses `=` instead of `==` for comparison **(1)**
Correction: `elif num == 0:` **(1)**
Syntax error 2: Line 6 is missing a colon after `else` **(1)**
Correction: `else:` **(1)**

Exam tip: Both errors here are syntax errors. The program would not run at all until both are fixed. Remember: = is assignment, == is comparison.

3. Robust programming

A **robust** program is one that can handle unexpected inputs or situations without crashing. Robust programs use validation to check input data and authentication to verify users.

3.1 Data validation

Validation is the process of checking that data entered is reasonable, sensible, and in the correct format. Validation does NOT check that data is correct — only that it is valid.

> **Important distinction:** Validation checks if data is **reasonable/sensible**. It cannot check if data is **accurate/true**.
> Example: An age of 25 is valid (reasonable for a human age), but validation cannot check if the person really is 25 years old.

Range check

A range check ensures data falls within an allowed minimum and maximum value.

```
score = int(input("Enter score (0-100): "))
while score < 0 or score > 100:
    print("Invalid. Score must be between 0 and 100.")
    score = int(input("Enter score (0-100): "))
print("Valid score entered:", score)
```

Length check

A length check ensures a string has at least a minimum number of characters (or at most a maximum).

```
password = input("Enter password (min 8 characters): ")
while len(password) < 8:
    print("Password too short. Must be at least 8 characters.")
    password = input("Enter password (min 8 characters): ")
print("Password accepted")
```

Presence check

A presence check ensures that data has been entered (the field is not empty).

```
name = input("Enter your name: ")
while name == "":
    print("Name cannot be empty.")
    name = input("Enter your name: ")
print("Hello,", name)
```

Combining validation checks

Multiple validation checks can be combined to ensure data meets several criteria:

```
age = input("Enter your age: ")
while not age.isdigit() or int(age) < 1 or int(age) > 120:
```

```
    print("Invalid. Enter a number between 1 and 120.")
    age = input("Enter your age: ")
age = int(age)
print("Your age is:", age)
```

Type checking with isdigit()

The `isdigit()` method checks if a string contains only digit characters (0-9). This is useful for validating that user input can be safely converted to an integer:

```
user_input = input("Enter a number: ")
if user_input.isdigit():
    number = int(user_input)
    print("You entered:", number)
else:
    print("That is not a valid number")
```

Note: `isdigit()` returns False for negative numbers (because of the minus sign) and decimals (because of the point). For GCSE, it's typically used for positive whole numbers.

Format checking

Some validation checks require specific formats. For example, checking an email contains "@":

```
email = input("Enter email: ")
while "@" not in email:
    print("Invalid email - must contain @")
    email = input("Enter email: ")
print("Email accepted:", email)
```

This is a simple format check — it doesn't guarantee the email is real, only that it contains the expected character.

> **Validation does not equal verification.** Validation checks that data is in the correct format and within acceptable ranges. It cannot check that data is actually true or accurate. An email passing validation (contains @) doesn't mean it's a real email address. A valid age (between 1-120) doesn't mean the user entered their real age.

3.2 Authentication

Authentication is the process of verifying that a user is who they claim to be. The most common form is username and password authentication.

Definition: Authentication is verifying the identity of a user, typically by checking credentials such as a username and password.

Simple authentication routine

A basic authentication routine compares entered credentials against stored values:

```
stored_username = "admin"
stored_password = "secret123"

username = input("Enter username: ")
password = input("Enter password: ")

if username == stored_username and password == stored_password:
    print("Login successful")
else:
    print("Invalid username or password")
```

Authentication with multiple attempts

A more robust authentication routine might allow multiple attempts:

```
stored_username = "admin"
stored_password = "secret123"
attempts = 0
max_attempts = 3
logged_in = False

while attempts < max_attempts and not logged_in:
    username = input("Enter username: ")
    password = input("Enter password: ")

    if username == stored_username and password == stored_password:
        logged_in = True
        print("Login successful")
    else:
        attempts = attempts + 1
        remaining = max_attempts - attempts
        print("Invalid credentials.", remaining, "attempts remaining")

if not logged_in:
    print("Account locked")
```

Authentication with multiple users

When there are multiple valid username/password combinations:

```
usernames = ["user1", "user2", "admin"]
passwords = ["pass1", "pass2", "adminpass"]

username = input("Enter username: ")
password = input("Enter password: ")

logged_in = False
for i in range(len(usernames)):
    if username == usernames[i] and password == passwords[i]:
        logged_in = True

if logged_in:
    print("Login successful")
else:
    print("Invalid username or password")
```

GCSE note. At GCSE, you only need to work with plain text passwords. In real systems, passwords are encrypted (hashed) before storage for security, but encryption is not required for AQA GCSE.

Worked Example 9 — Writing a range check validation

Question: Write Python code that asks the user to enter a number between 1 and 10 inclusive. The program should keep asking until a valid number is entered. [4 marks]

Model answer:

```
num = int(input("Enter a number (1-10): "))
while num < 1 or num > 10:
    print("Invalid. Number must be between 1 and 10.")
    num = int(input("Enter a number (1-10): "))
print("You entered:", num)
```

Marking points:

- Initial input **(1)**
- While loop with correct condition **(1)**
- Error message inside loop **(1)**
- Re-prompt inside loop **(1)**

Exam tip: Use a while loop (not if) to keep asking until valid. The condition checks for invalid data — the loop continues while data is invalid.

Worked Example 10 — Writing a length check validation

Question: Write Python code that asks for a password and validates that it is at least 8 characters long but no more than 20 characters. [4 marks]

Model answer:

```
password = input("Enter password (8-20 characters): ")
while len(password) < 8 or len(password) > 20:
    print("Invalid length. Password must be 8-20 characters.")
    password = input("Enter password (8-20 characters): ")
print("Password accepted")
```

Exam tip: Use len() to get string length. The while condition uses OR because we reject if too short OR too long.

Worked Example 11 — Writing a simple authentication routine

Question: Write Python code for a simple login system. The correct username is "student" and the correct password is "learn123". Display an appropriate message for successful or unsuccessful login. [4 marks]

Model answer:

```
correct_user = "student"
correct_pass = "learn123"

username = input("Enter username: ")
password = input("Enter password: ")

if username == correct_user and password == correct_pass:
    print("Login successful. Welcome!")
else:
    print("Invalid username or password")
```

Exam tip: Use AND because BOTH username AND password must match. Don't tell the user which one was wrong — this gives hackers information.

Worked Example 12 — Authentication with validation

Question: Write Python code that validates a username and password with the following rules: username must not be empty, password must be at least 6 characters. Then check if they match the stored credentials (username: "admin", password: "password"). [6 marks]

Model answer:

```
stored_user = "admin"
stored_pass = "password"

# Validate username (not empty)
username = input("Enter username: ")
while username == "":
    print("Username cannot be empty.")
    username = input("Enter username: ")

# Validate password (min 6 characters)
password = input("Enter password: ")
while len(password) < 6:
    print("Password must be at least 6 characters.")
    password = input("Enter password: ")

# Authenticate
if username == stored_user and password == stored_pass:
    print("Login successful")
else:
    print("Invalid credentials")
```

Exam tip: This combines validation (checking format) with authentication (checking identity). Validate first, then authenticate.

Worked examples

This page contains 12 worked examples covering all the key skills for testing and robust programming. The examples are integrated throughout the teaching sections above, demonstrating exam-style questions and model answers.

Worked examples summary

Use this table to find specific types of questions for revision.

Example	Topic	Question type	Marks
1	Identifying test data types	State type	4

Example	Topic	Question type	Marks
2	Selecting boundary data	State and justify	4
3	Completing a test plan	Complete table	4
4	Justifying test data choices	Give examples and justify	6
5	Identifying syntax errors	Identify and correct	2
6	Identifying logic errors	Identify and correct	2
7	Categorising errors	State error type	4
8	Finding multiple errors	Identify and correct	4
9	Range check validation	Write code	4
10	Length check validation	Write code	4
11	Simple authentication	Write code	4
12	Authentication with validation	Write code	6

Quick reference

Key definitions

Term	Definition (mark-scheme style)
Testing	Running a program with known inputs to check the outputs are correct and find errors.
Test data	Input values used to test a program.
Normal data	Typical, valid data within the expected range that the program should accept.
Boundary data	Values at the edges of the valid range (just inside and just outside boundaries).
Erroneous data	Invalid data that the program should reject (wrong type, out of range, etc.).
Syntax error	Code that breaks the rules of the programming language; program cannot run.
Logic error	Code that runs but produces incorrect results; the algorithm is wrong.
Validation	Checking that data entered is reasonable, sensible, and in the correct format.

Term	Definition (mark-scheme style)
Authentication	Verifying that a user is who they claim to be (e.g., username and password).

Test data selection guide

For a valid range of X to Y (inclusive):

- **Normal:** Any typical value between X and Y (e.g., middle of range)
- **Boundary:** X-1 (reject), X (accept), Y (accept), Y+1 (reject)
- **Erroneous:** Wrong data type, very large/small values, empty input

Common Python syntax errors

Error	Example	Fix
Missing colon	`if x > 5`	`if x > 5:`
Wrong comparison operator	`if x = 5:`	`if x == 5:`
Misspelled keyword	`whle x < 10:`	`while x < 10:`
Unclosed bracket	`print("Hello"`	`print("Hello")`
Unclosed string	`name = "John`	`name = "John"`
Wrong indentation	Code not indented after :	Indent with 4 spaces or tab

Validation code patterns

Range check pattern

```
value = int(input("Enter value: "))
while value < minimum or value > maximum:
    print("Invalid")
    value = int(input("Enter value: "))
```

Length check pattern

```
text = input("Enter text: ")
while len(text) < min_length:
```

```
    print("Too short")
    text = input("Enter text: ")
```

Presence check pattern

```
text = input("Enter text: ")
while text == "":
    print("Cannot be empty")
    text = input("Enter text: ")
```

Authentication pattern

```
username = input("Username: ")
password = input("Password: ")
if username == correct_user and password == correct_pass:
    print("Success")
else:
    print("Invalid")
```

Exam command words for this topic

Command word	What to do	Example question
Identify	Name or point out the feature/error	"Identify the syntax error in the code"
State	Give a brief answer without explanation	"State the type of test data"
Describe	Give key features and characteristics	"Describe what is meant by validation"
Explain	Give reasons or show how/why	"Explain why boundary data is used"
Write	Produce code that works	"Write a validation routine"
Correct	Fix the error and show the corrected version	"Correct the error in line 3"

Command word	What to do	Example question
Justify	Give reasons for your choice	"Justify your choice of test data"

Quick decision flowchart

Is it a syntax error or logic error?

- Does the program run at all? → **No** → Syntax error
- Does the program run at all? → **Yes, but wrong output** → Logic error

What type of test data is it?

- Is it within the valid range? → **Yes, typical value** → Normal data
- Is it at the exact edge of the valid range? → **Yes** → Boundary data
- Is it outside the valid range or wrong type? → **Yes** → Erroneous data

Typical pitfalls

Pitfall 1 — Confusing boundary and erroneous data.
Wrong: Saying 11 is erroneous when the range is 1—10.
Right: 11 is boundary data (just above upper boundary). Erroneous would be "abc" or -999.

Pitfall 2 — Missing boundary values.
Wrong: Testing only 1 and 10 for range 1—10.
Right: Test both sides of each boundary: 0, 1, 10, and 11.

Pitfall 3 — Confusing syntax and logic errors.
Wrong: Calling a wrong calculation a syntax error.
Right: If the program runs, it's a logic error. Syntax errors stop the program running.

Pitfall 4 — Using = instead of == in conditions.
Wrong: `if x = 5:`
Right: `if x == 5:` — single = is assignment, double == is comparison.

Pitfall 5 — Using if instead of while for validation.
Wrong: Using if to check input (only checks once).
Right: Use while to keep asking until valid input is entered.

Pitfall 6 — Wrong loop condition logic.
Wrong: `while num >= 1 and num <= 10:` (loops while valid)
Right: `while num < 1 or num > 10:` (loops while invalid)

Pitfall 7 — Confusing validation and authentication.
Wrong: Saying password checking is validation.
Right: Checking password format (length) is validation. Checking it matches stored password is authentication.

Pitfall 8 — Forgetting to re-prompt inside validation loop.
Wrong: Only asking for input once before the loop.
Right: Ask before loop AND inside loop, or use different loop structure.

Pitfall 9 — Using OR when AND is needed (or vice versa).
Wrong: `if user == "admin" or pass == "secret":`
Right: `if user == "admin" and pass == "secret":` — both must match.

Pitfall 10 — Not justifying test data choices.
Wrong: Just listing test data without explanation.
Right: Explain WHY each piece of data tests what it's supposed to test.

Assessment Hub | Long-Answer | Notes Index

Williams Physics Education — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional Qualifications. Free to use, no login required.

About & Contact • Privacy • Terms • Accessibility • Copyright • DMCA • YouTube

WPE is free for everyone. If our resources help you or your students, you can support the project here.

© 2026 Williams Physics Education. All rights reserved.